AI Pioneers

CCA-F | Module 5

Lab 5.2

Resilient Systems: Error Propagation & Large Codebase Exploration

---

Scenario: A Healthcare Claims Processing Pipeline (Intake → Validation → Adjudication)

Module	M5 — Context Management & Reliability
Lab type	Hands-on · Self-paced or Instructor-Led · Claude API (Python)
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.10+ · `anthropic` + `python-dotenv` · `ANTHROPIC_API_KEY`
Sections	S3 (Error Propagation in Multi-Agent Systems) , S4 (Context Management in Large Codebase Exploration)

Lab Objectives

By the end of this lab you will be able to:

- **Propagate errors cleanly through a multi-agent system** — wrap every subagent in a `StageResult(stage, ok, data, error)` envelope so failures cannot disappear. A subagent never raises into the coordinator; the coordinator decides how to react — log, retry, escalate — but it always knows what happened and why.
- **Use a disk-backed scratchpad as the single source of truth** — write every stage's finding to `scratchpad.json` as you go, so a human (or your future self) can audit exactly how each claim was handled. The scratchpad is also where checkpoint state lives — it's the file that makes the next objective possible.
- **Recover from a crash without redoing finished work** — on restart, the coordinator reads the scratchpad and skips claims already marked `done` or `failed`. A nightly batch that dies on claim #4,712 resumes at claim #4,713, never re-billing or re-processing what was already finished.

1. Overview

Multi-agent pipelines in production fail in two ways: silently, when a subagent raises an exception and the coordinator catches nothing useful, and catastrophically, when an hour into a nightly run the process dies and the restart re-does

work it already finished. This lab fixes both with three small, composable pieces. First, every subagent returns a StageResult envelope — ok / data / error — so the failure path is a first-class return value, not an exception escape hatch. Second, every stage's finding is written immediately to a disk-backed scratchpad — the audit trail and the checkpoint live in the same JSON file. Third, the coordinator reads the scratchpad on startup and skips claims already in done or failed state, so re-runs after a crash do exactly the unfinished work. The running example is a healthcare claims pipeline (intake → validation → adjudication) with one deliberately malformed claim so you can watch the failure propagate cleanly without taking the pipeline down. Everything runs as one main.py that's safe to Ctrl+C and restart at any time.

SN	Demo	What you will do	Core idea
Ex 1	**S3 — StageResult error envelope**	Wrap each subagent so it returns `StageResult(stage, ok, data, error)` — never raises. The coordinator inspects `result.ok` and reacts.	Failures become structured data the coordinator can act on, not exceptions that vanish.
Ex 2	**S4 — Disk-backed scratchpad**	Log every finding to `scratchpad.json` via `pad.log(claim_id, stage, payload)`; `mark_done` / `mark_failed` set the per-claim status.	Every step is auditable; the file doubles as the checkpoint.
Ex 3	**S4 — Crash-recovery skip rule**	On startup the coordinator reads the scratchpad and skips claims whose `status` is `done` or `failed`. A Ctrl+C is recoverable.	Re-runs do only unfinished work; the audit log doesn't lie about restarts.

2. Scenario

You are building an AI pipeline that processes insurance claims. Each claim runs through three specialist subagents: intake extracts structured fields from the raw claim narrative, validation checks policy rules (member active, procedure covered), and adjudication decides approve / hold-for-review / denied based on the amount. The pipeline runs over a list of claims nightly; an hour in, a subagent on claim #47 of 5,000 must not be able to take everything down or silently swallow a problem — that would translate into an unpaid or wrongly paid claim, which is exactly the failure mode regulators care about.

Your running example is three sample claims. CLM-001 and CLM-003 are well-formed and should sail through. CLM-002 is for an inactive member — the validation subagent must catch it, return a StageResult with ok=False and a structured error, and the coordinator must record the failure and move on without crashing. Then you'll demonstrate crash recovery by re-running the pipeline: every already-handled claim is skipped, and the scratchpad still holds the complete audit trail from the first run.

You will:

- The `StageResult` dataclass is provided; implement the three subagent functions — `run_intake`, `run_validation`, `run_adjudication` — each of which returns a `StageResult` and never raises into the coordinator.
- Build `Scratchpad` — a small JSON-on-disk store keyed by `claim_id` with `log()`, `mark_done()`, `mark_failed()`, and `status()` methods. Every mutation flushes to disk.

- Wire the coordinator in `main.py` so it walks each claim through all three stages, logs every finding, marks the final status, and on restart skips anything already `done` or `failed`.

The pipeline at a glance

```
INTAKE  ->  VALIDATION  ->  ADJUDICATION
each stage  ->  StageResult(stage, ok, data, error)
every step  ->  pad.log(claim_id, stage, payload)  ->  scratchpad.json
on restart  ->  skip if pad.status(claim_id) in ('done','failed')
```

Why these three together?

Each piece is useless without the other two. The StageResult envelope makes failures explicit — but if you never write them down, you cannot audit them and you cannot restart cleanly. The scratchpad logs everything to disk — but if subagents raise exceptions instead of returning failures, half the entries are missing and the file lies about what actually happened. Crash recovery reads the scratchpad on startup — but only if the scratchpad accurately reflects which claims finished, which requires both the structured-error contract and the per-mutation flush. Together they give you a pipeline that fails loudly into a file, survives crashes, and resumes exactly where it stopped. Drop any one and you re-introduce a silent-failure or duplicate-work mode that costs real money in claims processing.

3. Pre-requisites

3.1 Skilljar Videos — Completed in Earlier Modules

Course	Lessons to watch	Covers
Introduction to Subagents	Designing effective subagents · Using subagents effectively	**S3**
Claude Code in Action	Adding context · Controlling context	**S4**

Module 5 introduces no new Skilljar courses — it builds on lessons you completed in Modules 1 and 2. Revisit the specific lessons above before the session; the instructor will not re-teach them. Sections S5 and S6 (not covered in this lab) have no dedicated Skilljar lesson at all; they are built in their respective instructor-led labs.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
- Unzip the lab bundle and enter it:

```
unzip lab_5_2_resilient_systems.zip
cd lab_5_2_resilient_systems
```

- Create a Python environment and install both dependencies:

```
python -m venv .venv && source .venv/bin/activate   # Windows:
.venv\Scripts\activate
pip install -r requirements.txt                     # anthropic + python-dotenv
```

- Add your API key (the lab uses `python-dotenv` — paste your key into `.env`):

```
cp .env.example .env          # paste your key into .env
# .env content:
# ANTHROPIC_API_KEY=sk-ant-your-key-here
# MODEL_NAME=claude-sonnet-4-5
```

- Confirm your setup: run python main.py. Until you complete the three TODOs it stops at the first one with a clear message — that still confirms the modules import and your dependencies loaded. With the TODOs done it runs end-to-end on the three sample claims (intake calls the model):

```
python main.py
```

Model, cost & safety

The script reads the model from `MODEL_NAME` (default `claude-sonnet-4-5` — the intake subagent is the only stage that calls the API; validation and adjudication are deterministic Python). Cost is roughly one short JSON extraction per claim — minimal. The mock data in `sample_claims.py` is fictional (synthetic member IDs M-501 / M-777, CPT codes from a tiny test set) — no PHI, no real medical records. To reset, delete `scratchpad.json` by hand; the script never deletes it (that's the user's manual switch).

4. Demos

Open `main.py`. Each demo has one TODO to complete (see each file's header); the coordinator is short. The supporting modules (`agents.py`, `scratchpad.py`, `sample_claims.py`) implement the three techniques. Running `python main.py` exercises all three demos in one pass — failure propagation on CLM-002, the growing scratchpad audit, and (on a second run) crash-recovery skipping. The three demos are conceptual; in code they are interleaved into one coordinator loop.

Demo 1: Propagate errors with StageResult (Error Propagation, S3) ~15 min

Background

A subagent that raises an exception into the coordinator is a subagent that disappears: the coordinator catches a generic exception, logs a vague message, and the structured details — which stage, which input, what kind of failure — are lost. The fix is a **typed envelope** every subagent must return. `StageResult` carries the stage name, an `ok` boolean, an optional `data` payload on success, and an optional `error` string on failure. The subagent never raises; the coordinator inspects `result.ok` and decides what to do.

Task

Open `agents.py` and implement its three subagents. Study the `StageResult` dataclass, then `run_intake` (which calls Claude and must catch any exception), `run_validation` (which checks policy rules and returns structured errors), and `run_adjudication` (deterministic decision). Notice that the intake function has a bare `except Exception` — that is deliberate: subagents NEVER raise into the coordinator.

Step 1 — The StageResult dataclass

A four-field dataclass with an `asdict`-based serializer so the scratchpad can log it. `data` and `error` are both optional — exactly one is populated per call:

```
from dataclasses import dataclass, asdict
from typing import Any

@dataclass
class StageResult:
    """Envelope every subagent must return. Failures are first-class."""
    stage: str
    ok: bool
    data: Any = None
    error: str | None = None

    def to_dict(self) -> dict:
        return asdict(self)
```

Step 2 — Intake: catch everything, return a StageResult

The intake subagent is the only one that calls the API, so it is the most likely to fail (timeouts, malformed JSON, rate limits). The `try / except Exception` wrap is the only place in the file that uses `except` — and it always converts the exception into a `StageResult(ok=False, error=...)` instead of re-raising:

```
def run_intake(claim: dict) -> StageResult:
    """Call Claude to produce a clean structured summary of the claim."""
    try:
        response = _client.messages.create(
            model=MODEL_NAME, max_tokens=400, system=INTAKE_SYSTEM,
            messages=[{"role": "user", "content": json.dumps(claim)}],
        )
        text = "".join(b.text for b in response.content if b.type ==
"text").strip()
        # be forgiving about ```json fences ...
```

```
        parsed = json.loads(text)
        return StageResult(stage="intake", ok=True, data=parsed)
    except Exception as exc:
        return StageResult(stage="intake", ok=False,
                           error=f"{type(exc).__name__}: {exc}")
```

Step 3 — Validation: structured failures with a named error code

No try/except needed — the validation rules are deterministic Python. Failures still come back as `StageResult(ok=False)` with a short, machine-readable `error` code so the coordinator can branch on it:

```
def run_validation(claim: dict) -> StageResult:
    """Check policy rules. Fail fast and loudly."""
    if not claim.get("member_active", False):
        return StageResult(stage="validation", ok=False,
                           error="member_not_active")
    if claim.get("procedure_code") not in COVERED_PROCEDURES:
        return StageResult(stage="validation", ok=False,

error=f"procedure_not_covered:{claim.get('procedure_code')}")
    return StageResult(stage="validation", ok=True, data={"checks_passed": True})
```

Step 4 — Run and watch CLM-002 fail cleanly

```
python main.py
```

Expected: CLM-001 and CLM-003 print `intake...ok  validation...ok  adjudication...ok`. CLM-002 prints `intake...ok  validation...FAIL (member_not_active)` and the coordinator records the failure without taking the pipeline down. The other two claims still finish.

> **What good looks like**
>
> The malformed claim surfaces a specific, named error (`member_not_active` — not a stack trace) and the run completes with a clean summary line: `done: 2  failed: 1  skipped: 0`. No subagent has raised into the coordinator. If anything looks like a stack trace in the printed output, a subagent forgot its envelope.

Reflection Questions

- Why force every subagent to return a `StageResult` rather than letting exceptions bubble up?
- The intake error string is `f"{type(exc).__name__}: {exc}"`, validation errors are short codes like `"member_not_active"`. When is each style appropriate?
- How would you extend `StageResult` to support a retry-local-then-escalate policy (per the Module 5 plan), and what would the coordinator's logic look like?

Demo 2: Scratchpad as audit trail (Context Management, S4) ~15 min

Background

Holding pipeline state in memory is fine until something crashes. The scratchpad pattern keeps that state on disk in a single JSON file, keyed by `claim_id`, with two responsibilities: (1) record every finding for audit (intake output, validation result, adjudication decision), and (2) hold each claim's `status` (one of `new`, `in_progress`, `done`, `failed`). One file does both jobs because they share the same key — and writing to it after every mutation is cheap at lab scale.

Task

Open `scratchpad.py` and implement its four mutating methods (log, mark_done, mark_failed, _flush); __init__ and status() are provided. Read the `Scratchpad` class: a `path`, an in-memory `_data` dict loaded from disk on init, and four mutating methods that each call `_flush()`. Notice the JSON-decode fallback at the top — if the file is corrupt, the class starts fresh but does NOT delete the existing file (alerting belongs in a real system).

Step 1 — Construct and load existing state

```
class Scratchpad:
    """A tiny JSON-on-disk store keyed by claim_id."""

    def __init__(self, path: str = "scratchpad.json"):
        self.path = Path(path)
        self._data: dict = {}
        if self.path.exists():
            try:
                self._data = json.loads(self.path.read_text())
            except json.JSONDecodeError:
                # Corrupted file — start fresh but don't delete it.
                self._data = {}
```

Step 2 — Log a finding (every mutation flushes)

The `log` method appends a `{stage, payload}` record to that claim's `findings` list and writes the whole file back. **Every mutation flushes** — that is the contract that makes crash recovery work:

```
def log(self, claim_id: str, stage: str, payload) -> None:
    """Record what happened in a given stage for a given claim."""
    entry = self._data.setdefault(claim_id,
                                  {"status": "in_progress", "findings": []})
    entry["findings"].append({"stage": stage, "payload": payload})
    self._flush()

def _flush(self) -> None:
    self.path.write_text(json.dumps(self._data, indent=2))
```

Step 3 — Mark terminal status; coordinator decides which

```
def mark_done(self, claim_id: str) -> None:
    self._data[claim_id]["status"] = "done"
    self._flush()

def mark_failed(self, claim_id: str, reason: str) -> None:
    entry = self._data.setdefault(claim_id, {"findings": []})
    entry["status"] = "failed"
    entry["failure_reason"] = reason
    self._flush()
```

Step 4 — Run, then inspect scratchpad.json

```
python main.py
cat scratchpad.json    # full audit trail
```

Expected: after a clean run, `scratchpad.json` contains three entries (one per claim). Each has a `status`, a `findings` array with one record per stage that ran, and (for CLM-002) a `failure_reason` naming the stage and the structured error. Open the file in your editor — it is human-readable and exactly the kind of artifact an auditor would ask for.

What good looks like

Every stage that ran for every claim is in the file, in order. Successful claims end with `"status": "done"`; the failed claim ends with `"status": "failed"` and a `failure_reason` that names the stage and the error code. The validation entry for CLM-002 carries `ok: false` and the same error string. No exceptions, no missing entries.

Reflection Questions

- Why does `_flush()` run after EVERY mutation, instead of batching writes? When would batching be the right trade-off?
- The corrupted-file branch starts fresh but does NOT delete the bad file. Why is keeping it the safer default in a real system?
- How would the scratchpad need to change to support concurrent coordinators (e.g. two workers consuming the same claims list)?

Demo 3: Crash recovery — resume on rerun (Context Management, S4) ~15 min

Background

Long batches die. A laptop sleeps, a VM evicts, somebody hits Ctrl+C. The scratchpad turns that from a re-do-everything problem into a resume-where-we-stopped problem: on startup, the coordinator reads the file, and for each claim already in `done` or `failed` state, it skips. Only claims in `new` or `in_progress` get processed. Because every mutation flushed to disk, `in_progress` claims will simply be re-run from the start (idempotent given the deterministic stages); `done` / `failed` claims are untouched.

Task

Complete process_claim() and the skip rule in the `main()` coordinator loop of `main.py`. For each claim it asks `pad.status(claim_id)` first; only if the status is not `done` or `failed` does it call `process_claim`. Then run the lab twice and watch the difference.

Step 1 — The coordinator's skip rule

```
for claim in CLAIMS:
    status = pad.status(claim["claim_id"])
    # ----- Crash recovery: skip claims already finished. -----
    if status in ("done", "failed"):
        print(f"[CLAIM {claim['claim_id']}] (already {status}, skipping)")
        skipped_count += 1
        continue
    final = process_claim(claim, pad)
    if final == 'done':
        done_count += 1
    else:
        failed_count += 1
```

Step 2 — process_claim: log every stage, mark terminal status

Each per-claim walk runs the three stages in order. After every stage, the result is logged. On the first `ok=False`, the claim is marked `failed` with the offending stage name and error string. Only after all three stages succeed is the claim marked `done`:

```
def process_claim(claim: dict, pad: Scratchpad) -> str:
    claim_id = claim["claim_id"]
    stages = [
        ("intake", run_intake),
        ("validation", run_validation),
        ("adjudication", run_adjudication),
    ]
    for stage_name, fn in stages:
        result = fn(claim)
        pad.log(claim_id, stage_name, result.to_dict())
        if not result.ok:
            pad.mark_failed(claim_id, f"{stage_name}: {result.error}")
            return "failed"
    pad.mark_done(claim_id)
    return "done"
```

Step 3 — Run twice; first does the work, second does nothing

```
python main.py    # first run — processes all three claims
python main.py    # second run — every claim is 'already done/failed, skipping'
```

Expected second-run summary: `done: 0  failed: 0  skipped: 3`. To simulate a real crash, press Ctrl+C mid-run between two claims and re-run — claims processed before the Ctrl+C are skipped; the ones after pick up cleanly. To

start over, delete `scratchpad.json` by hand (the lab never deletes it from code — that is the user's manual reset switch).

What good looks like

Second run prints three `(already done, skipping)` / `(already failed, skipping)` lines and a summary of `done: 0  failed: 0  skipped: 3`. A Ctrl+C-then-restart shows the first run's finished claims as skipped and continues from the first `new` or `in_progress` claim. If a finished claim re-runs, the per-mutation flush isn't happening; check `_flush()`.

Reflection Questions

- Why does the coordinator skip on `done` AND `failed` — wouldn't you want failed claims to be retried automatically?
- An `in_progress` claim gets re-run from stage 1 on restart. When is that safe, and when does it cause double-billing or other side-effect problems?
- The lab never deletes `scratchpad.json` from code — the user resets manually. What goes wrong if you make the reset automatic ("delete the file if it's older than 24 hours")?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	Why must every subagent return a `StageResult` instead of raising — and what is the one place in the codebase where an exception is allowed to be caught?
2	What does the scratchpad do beyond audit? Why is it the right place for the checkpoint state?
3	On restart, which claim statuses cause a skip and which cause a re-run? How does that interact with the `_flush()` contract?
4	How do you distinguish a timeout failure from a valid 'no data found' result in this design? Where would that distinction live?
5	How do the three pieces (envelope, scratchpad, skip-on-restart) reinforce each other? Drop any one and explain what breaks.

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Letting subagents raise into the coordinator.	A bare `except Exception` in the coordinator collapses every failure into a vague "something went wrong" — you lose the stage, the input, the type. Wrap every subagent so it returns `StageResult(ok=False, error=...)`.
Holding pipeline state in memory only.	A crash takes everything with it. Write each finding to disk as it happens. Memory is for speed; disk is for survival.
Batching writes to the scratchpad.	A crash between batches is exactly when the scratchpad needs to be honest. At lab scale a per-mutation flush is free; at production scale, move to SQLite with proper transactions — but don't drop the **write-immediately** contract.
Deleting scratchpad.json from code.	"Auto-reset on staleness" sounds tidy and is a disaster: a node-clock skew or a paused job suddenly throws away the audit trail. Keep the reset manual.
Retrying everything on restart.	If you re-run already-`done` claims, you risk re-billing or duplicate side effects. Skip on `done` AND `failed`; let humans decide which failures to retry manually.
Stringifying errors instead of typing them.	Codes like `"member_not_active"` let the coordinator branch (retry? alert? human handoff?). Free-text messages from the model belong in `data`, not `error`.
Skipping the scratchpad for "small" runs.	Every run that does work in production should leave an audit trail. The scratchpad is cheap; debugging a silent batch is not.

6. Quick Reference

6.1 StageResult envelope and a subagent contract

```
from dataclasses import dataclass, asdict

@dataclass
class StageResult:
    stage: str
    ok: bool
    data: any = None
    error: str | None = None
    def to_dict(self): return asdict(self)

# Every subagent MUST return StageResult and MUST NOT raise.
def run_stage(claim) -> StageResult:
    try:
        return StageResult(stage='example', ok=True, data=...)
    except Exception as exc:
        return StageResult(stage='example', ok=False,
                           error=f'{type(exc).__name__}: {exc}')
```

6.2 Scratchpad — disk-backed audit + checkpoint

```
class Scratchpad:
    def __init__(self, path='scratchpad.json'):
        self.path = Path(path)
        self._data = {}
        if self.path.exists():
            try: self._data = json.loads(self.path.read_text())
            except json.JSONDecodeError: self._data = {}

    def status(self, cid):  return self._data.get(cid, {}).get('status', 'new')
    def log(self, cid, stage, payload):
        e = self._data.setdefault(cid, {'status':'in_progress','findings':[]})
        e['findings'].append({'stage': stage, 'payload': payload})
        self._flush()
    def mark_done(self, cid):   self._data[cid]['status']='done';   self._flush()
    def mark_failed(self, cid, reason):
        e = self._data.setdefault(cid, {'findings':[]})
        e['status']='failed'; e['failure_reason']=reason; self._flush()
    def _flush(self):  self.path.write_text(json.dumps(self._data, indent=2))
```

6.3 The coordinator loop (skip + walk + log + mark)

```
pad = Scratchpad('scratchpad.json')
for claim in CLAIMS:
    if pad.status(claim['claim_id']) in ('done', 'failed'):
        continue                                # crash-recovery skip
    cid = claim['claim_id']
    for name, fn in [('intake', run_intake),
                     ('validation', run_validation),
                     ('adjudication', run_adjudication)]:
        r = fn(claim)
        pad.log(cid, name, r.to_dict())         # audit trail
        if not r.ok:
            pad.mark_failed(cid, f'{name}: {r.error}')
            break
    else:
            pad.mark_done(cid)                  # all three stages ok
```

6.4 Three pieces, one contract

```
# All three are required for resilience.
#
#   envelope     -> StageResult(ok, data, error)  (Demo 1, S3)
#   audit/check  -> pad.log + pad.mark_*           (Demo 2, S4)
#   skip-restart -> pad.status() in ('done','failed') (Demo 3, S4)
```

```
#
# Drop the envelope -> the scratchpad lies (missing entries).
# Drop the scratchpad -> crashes lose all progress.
# Drop skip-restart -> re-runs duplicate work and re-bill claims.
```

6.5 File Inventory

File	Role	Purpose
`main.py`	**entry**	The coordinator. Walks each claim through the three stages, logs every finding, marks the terminal status, and on restart skips already-finished claims.
`agents.py`	**D 1 (S3)**	The `StageResult` dataclass and the three subagents (`run_intake`, `run_validation`, `run_adjudication`). Intake calls Claude; the others are deterministic.
`scratchpad.py`	**D 2/3 (S4)**	The `Scratchpad` class: per-mutation flush of an audit-log + checkpoint JSON file (`status`, `findings`, `failure_reason`).
`sample_claims.py`	**data**	Three mock claims (CLM-001 passes, CLM-002 is for an inactive member, CLM-003 passes) plus the tiny `COVERED_PROCEDURES` policy set.
`.env.example, requirements.txt`	**setup**	API key template (with optional `MODEL_NAME` override) and the two pinned dependencies (`anthropic`, `python-dotenv`).

6.6 Further Reading

- Anthropic API overview: https://docs.claude.com/en/api/overview
- Tool use guide: https://docs.claude.com/en/docs/build-with-claude/tool-use
- Skilljar (Module 1) — **Introduction to Subagents**: Designing effective subagents; Using subagents effectively (S3)
- Skilljar (Module 2) — **Claude Code in Action**: Adding context; Controlling context (S4)